Data Quest

Dominando las Colecciones de Python

Resumen: ¡Viaja por el reino digital como ingeniero de datos! Domina las potentes estructuras de datos mientras construyes y procesas datos de juego.

Versión: 3.0

Contenidos

I Prólogo 2

II Instrucciones de IA 3

III Introducción 5

IV Instrucciones Comunes 6

IV.1 Reglas Generales 6

IV.2 Directrices Adicionales 6

V Ejercicio 0: Misión de Comando 7

VI Ejercicio 1: Triturador de Puntuaciones 9

VII Ejercicio 2: Rastreador de Posiciones 11

VIII Ejercicio 3: Cazador de Logros 13

IX Ejercicio 4: Maestro de Inventario 15

X Ejercicio 5: Mago de Transmisiones 17

XI Ejercicio 6: Álchemist de Datos 19

XII Entrega y Envío 21

Capítulo I Prólogo

En los primeros días de crecimiento de Twitter, escalar no consistía simplemente en agregar más servidores. Se trataba de corregir decisiones diminutas que de repente importaban a gran escala. Un patrón que apareció más de una vez fue usar estructuras de datos secuenciales simples (como listas o arreglos) porque eran fáciles y perfectamente bien para un tráfico reducido. Luego la base de usuarios explotó. Operaciones como “¿esto ya está presente?” que antes tomaban microsegundos comenzaron a ejecutarse millones de veces por segundo, convirtiéndose silenciosamente en cuellos de botella de rendimiento. Nada estaba lógicamente mal: el código era limpio y correcto, pero el contenedor equivocado convirtió el tiempo lineal en un dolor de cabeza de producción. Cambiar a estructuras basadas en hash a menudo solucionaba el problema de forma casi mágica. Es un gran recordatorio de que, a gran escala, tu estructura de datos es tu algoritmo.

Capítulo II

Instrucciones de IA

Contexto

Durante tu viaje de aprendizaje, la IA puede ayudar con muchas tareas diferentes. Tómate el tiempo para explorar las variadas capacidades de las herramientas de IA y cómo pueden apoyar tu trabajo. Sin embargo, aborda siempre estas herramientas con precaución y evalúa críticamente los resultados. Ya sea código, documentación, ideas o explicaciones técnicas, nunca puedes estar completamente seguro de que tu pregunta está bien planteada o de que el contenido generado es preciso. Tus compañeros son un recurso valioso para ayudarte a evitar errores y puntos ciegos.

Mensaje principal

- Usa la IA para reducir tareas repetitivas o tediosas.
- Desarrolla habilidades de prompting, tanto de codificación como no codificación que beneficiarán tu futura carrera.
- Aprende cómo funcionan los sistemas de IA para anticipar y evitar riesgos comunes, sesgos y cuestiones éticas.
- Sigue desarrollando habilidades técnicas y de alto impacto trabajando con tus compañeros.
- Utiliza solo contenido generado por IA que entiendas completamente y de cuya responsabilidad puedas asumir.

Reglas para el

- Debes tomarte el tiempo para explorar las herramientas de IA y entender cómo funcionan, para que las uses de forma ética y reduzcas sesgos potenciales.
- Debes reflexionar sobre tu problema antes de solicitar indicaciones: esto te ayuda a escribir indicaciones más claras, detalladas y relevantes usando vocabulario preciso.
- Debes desarrollar el hábito de revisar, cuestionar y probar sistemáticamente todo lo generado por IA.
- Siempre busca la revisión de pares no confíes únicamente en tu propia validación.

Resultados de la fase:

- Desarrollar habilidades de prompting tanto de uso general como específicas de dominio.
- Aumenta tu productividad con el uso efectivo de herramientas de IA.
- Continúa fortaleciendo el pensamiento computacional, la resolución de problemas, la adaptabilidad y la colaboración.

Comentarios y ejemplos:

- Con frecuencia encontrarás situaciones como exámenes, evaluaciones y más, donde debes demostrar una comprensión real. Prepárate, sigue desarrollando tanto tus habilidades técnicas como interpersonales.
- Explicar tu razonamiento y debatir con pares a menudo revela lagunas en tu comprensión. Haz del aprendizaje entre pares una prioridad.
- Las herramientas de IA a menudo no tienen tu contexto específico y tienden a proporcionar respuestas genéricas. Tus pares, que comparten tu entorno, pueden ofrecer ideas más relevantes y precisas.
- Donde la IA tiende a generar la respuesta más probable, tus pares pueden ofrecer perspectivas alternativas y matices valiosos. Confía en ellos como un punto de control de calidad.

✓ Buenas prácticas:

> Le pregunto a la IA: “¿Cómo pruebo una función de ordenación?” Me da algunas ideas. Las pruebo y reviso los resultados con un par. Refinamos el enfoque juntos.

✗ Malas prácticas:

> Pido a la IA que escriba toda una función, la copio-pego en mi proyecto. Durante la evaluación entre pares, no puedo explicar qué hace ni por qué. Pierdo credibilidad, y fallo mi proyecto.

✓ Buenas prácticas:

> Uso la IA para ayudar a diseñar un analizador. Luego recorro la lógica con un par. Detectamos dos errores y lo reescribimos juntos: mejor, más limpio y completamente entendido.

✗ Malas prácticas:

> Dejo que Copilot genere una parte clave de mi proyecto. Compila, pero no puedo explicar cómo maneja los pipes. Durante la evaluación, no puedo justificarlo y fallo mi proyecto.

Capítulo III

Introducción

Bienvenido de nuevo al reino digital, ingeniero de datos.

Tu viaje por los fundamentos de Python te ha preparado bien. Has dominado la sintaxis básica que impulsa jardines digitales, construido jerarquías de clases robustas que modelan sistemas del mundo real y aprendido a manejar lo inesperado con una gestión de excepciones elegante.

gestión. Ahora, estás listo para abordar el núcleo de la ingeniería de **colecciones y** datos: estructuras de datos que explorarás en un entorno de juego.

Imagínalo: en 1980, todo el estado del juego de Pac-Man, cada punto, posición de los fantasmas y puntuación, cabía en solo 16 KB de RAM. ¡Los programadores tuvieron que ser magos de la eficiencia! Descubrieron que organizar datos no era solo ahorrar memoria: era desbloquear la magia del juego. Hoy, Fortnite procesa más de 10 millones de jugadores concurrentes, cada uno generando miles de puntos de datos por segundo. Mismos principios, patio de juegos más grande.

Los tipos de colección de Python están diseñados para varios casos de uso, y cada uno tiene sus características: listas (ordenadas, indexadas, expansibles), tuplas (ordenadas, inmutables, hashables), conjuntos (colecciones no ordenadas de elementos únicos) y diccionarios (pares clave-valor).

Encima de estos vienen los generadores y comprensiones, que añaden sintaxis y comportamientos potentes.

En esta misión, construirás los componentes para apoyar una plataforma de análisis de juegos. Cada ejercicio desbloquea un nuevo tipo de datos y, al final, manejarás colecciones de Python como un ingeniero de datos.

Comenzando con este proyecto, y cuando se introduzca adecuadamente por medio de un ejercicio, podrás usar cada nueva estructura de datos de Python y todos sus métodos de clase asociados.

Capítulo IV

Instrucciones comunes

IV.1 Reglas Generales

- Tu proyecto debe estar escrito en Python 3.10 o posterior.
- Tu proyecto debe adherirse al estándar de codificación flake8.
- Todas las funciones y métodos deben incluir anotaciones de tipo; verifica esto usando mypy.
- Tus funciones deben manejar las excepciones de forma elegante para evitar fallos.
- Para este proyecto, necesitarás acceso a parámetros de línea de comandos. Harás esto utilizando el módulo sys a través del mecanismo de importación. Las importaciones se tratarán con más detalle en un proyecto futuro.
- No se permiten operaciones de entrada/salida de archivos. Todos los datos deben procesarse en memoria o mediante argumentos de la línea de comandos.
- Concéntrate en demostrar claramente los patrones de uso de colecciones.
- Muestra tanto operaciones básicas como técnicas avanzadas para cada estructura de datos.

Los siguientes tipos estándar están permitidos, junto con todos sus métodos y constructores asociados: str, int, float.

IV.2 Directrices adicionales

- Envía tu trabajo al repositorio de Git asignado.
- Solo se evaluará el contenido de este repositorio.

Capítulo V

Ejercicio 0: Búsqueda de Comandos

	Ejercicio0
ft_command_quest	
Directorio: ex0/	
Archivos a enviar: ft_command_quest.py	
Autorizado: import sys, sys.argv, len(), print()	

¡Bienvenido, Aventurero de Datos! Toda misión épica comienza entendiendo tus herramientas. En el reino digital, los programas deben recibir instrucciones del mundo exterior. Tu primera misión es descubrir cómo los programas pueden recibir mensajes de sus usuarios.

Ahora es el momento de presentar las listas. Antes de construir tus propias listas, manipulemos una lista que ya existe: los parámetros de línea de comandos, disponibles a través del módulo sys. La estructura es similar a la de C: una matriz de cadenas. Explora cómo acceder y manipular los elementos de la lista.

Crea un script sencillo que muestre los datos recibidos como parámetros de la línea de comandos. Imita el ejemplo a continuación.

```
$> python3 ft_command_quest.py
=== Misión de Comandos ===
Nombre del programa: ft_command_quest.py
¡No se proporcionaron argumentos!
Total de argumentos: 1

$> python3 ft_command_quest.py hola mundo 42
=== Misión de Comandos ===
Nombre del programa: ft_command_quest.py
Argumentos recibidos: 3
Argumento 1: hola
Argumento 2: mundo
Argumento 3: 42
Total de argumentos: 4

$> python3 ft_command_quest.py "Data Quest"
=== Misión de Comandos ===
Nombre del programa: ft_command_quest.py
Argumentos recibidos: 1
Argumento 1: Data Quest
Total de argumentos: 2
```

Simplemente usa “import sys” en la parte superior de tu script para acceder a la lista sys.argv.

Hay varias formas de evitar imprimir nuevamente el nombre del programa junto con los argumentos. Prepárese para discutir soluciones alternativas durante la evaluación.

Capítulo VI

Ejercicio 1: Desarrollador de puntuaciones

	Ejercicio1
ft_score_analytics	
Directorio: ex1/	
Archivos para enviar: ft_score_analytics.py	
Autorizado: import sys, sys.argv, len(), sum(), max(), min(), print()	

Resumen de la misión: Ahora que has dominado la comunicación de comandos, ¡es hora de una limpieza de datos! De hecho, el usuario que envía comandos al programa es humano y puede cometer errores.

Este ejercicio requiere que uses listas para almacenar puntuaciones y bloques try/except para manejar entradas inválidas de forma elegante (p. ej., cuando un usuario proporciona valores no numéricos).

Obtendrás puntuaciones de juegos como parámetros de línea de comandos. Debes:

- Procesar los argumentos de la línea de comandos
- Manejar los diversos casos erróneos (sin argumentos, valores no numéricos) con mensajes apropiados
- Crear una nueva lista para almacenar y organizar las puntuaciones
- Calcular algunas estadísticas básicas que harían feliz a cualquier jugador (número, total, promedio, máximo, mínimo y rango)
- Haz que la salida luzca lo suficientemente genial para impresionar a tus amigos jugadores (puedes imitar el ejemplo nuevamente)
- Si se proporcionan entradas válidas e inválidas a través de la línea de comandos, descarta las inválidas y continúa con las entradas válidas restantes a menos que no quede ninguna.

```
$> python3 ft_score_analytics.py 1500 2300 1800 2100 1950
=== Análisis de Puntuaciones de Jugadores ===
Puntuaciones procesadas: [1500, 2300, 1800, 2100, 1950]
Total de jugadores: 5
Puntuación total: 9650
Puntuación media: 1930.0
Puntuación más alta: 2300
Puntuación más baja: 1500
Rango de puntuación: 800

$> python3 ft_score_analytics.py
=== Análisis de Puntuaciones de Jugadores ===
No se proporcionaron puntuaciones. Uso: python3 ft_score_analytics.py <score1> <score2> ...

$> python3 ft_score_analytics.py ab ac
=== Análisis de Puntuaciones de Jugadores ===
Parámetro no válido: 'ab'
Parámetro no válido: 'ac'
No se proporcionaron puntuaciones. Uso: python3 ft_score_analytics.py <score1> <score2> ...
```

Capítulo VII

Ejercicio 2: Rastreador de posición

	Ejercicio2
ft_sistema_coordenadas	
Directorio: ex2/	
Archivos a entregar: ft_coordinate_system.py	
Autorizado: importar math, math.sqrt(), input(), round(), print()	

¡Nivel superior! ¡Hora de dominar coordenadas 3D! ¿Recuerdas jugar juegos donde teleport a ubicaciones específicas en un mundo 3D? ¿O cuando necesitas encontrar la distancia entre dos puntos en el espacio 3D? Eso es exactamente lo que estamos construyendo!

Este ejercicio requiere el uso de tuplas para almacenar coordenadas 3D (x, y, z).

Primero, escribe una función get_player_pos() que:

- Pide al usuario las nuevas coordenadas del jugador en el formato x,y,z
- Maneja entradas incorrectas
- Reintenta hasta que se proporcione un conjunto válido de coordenadas
- Devuelve una tupla que contiene las coordenadas 3D actuales del jugador

Luego tu código hará:

- Obtén un primer conjunto de coordenadas
- Muestra la tupla y luego muestra cada coordenada por separado
- Calcula la distancia al centro 3D (0, 0, 0) (ver más abajo)
- Obtén un nuevo conjunto de coordenadas
- Calcula la distancia entre el segundo y el primer conjunto de coordenadas

Fórmula de la Distancia: Para calcular la distancia entre dos puntos 3D, usamos la Eu-
fórmula de distancia euclidiana $-x1)2 + (y2-y1)2 + (z2-z1)2$.

Para puntos (x1, y1, z1) y (x2, y2, z2), la distancia es math.sqrt((x2-x1)**2 + (y2-y1)**2 + (z2-z1)**2). ¡Esta es solo la extensión 3D del teorema de Pitágoras!

```
$> python3 ft_coordinate_system.py
=== Sistema de Coordenadas del Juego ===

Obtén un primer conjunto de coordenadas
Ingresa nuevas coordenadas como números flotantes en formato 'x,y,z': hello world
Sintaxis inválida
Ingresa nuevas coordenadas como números flotantes en formato 'x,y,z': 1.0 , 2.5, 3.0
Obtuvimos la primera tupla: (1.0, 2.5, 3.0)
Incluye: X=1.0, Y=2.5, Z=3.0
Distancia al centro: 4.0311

Obtén un segundo conjunto de coordenadas
Ingresa nuevas coordenadas como números flotantes en formato 'x,y,z': 4,abc,5
Error en el parámetro 'abc': no se pudo convertir la cadena a flotante: 'abc'
Ingresa nuevas coordenadas como números flotantes en formato 'x,y,z': 4,5,6
Distancia entre los 2 conjuntos de coordenadas: 4.9244
```

Basta con usar ’import math’ en la parte superior de tu script para usar math.sqrt().

Las tuplas son como datos escritos en piedra. Una vez creadas, no cambiarán.

Capítulo VIII

Ejercicio 3: Cazador de logros

	Ejercicio3
ft_achievement_tracker	
Directorio: ex3/	
Archivos a enviar: ft_achievement_tracker.py	
Autorizado: len(), print(), import random, random.*, set(), set.union(), set.intersection(), set.difference()	

¡Logro desbloqueado! ¡Hora de construir el sistema de logros más increíble de todos! Tú sabes lo satisfactorio que es cuando desbloqueas ese logro raro? Ahora estás construyendo el sistema que los rastrea a todos.

Este ejercicio requiere el uso de conjuntos para almacenar logros únicos y realizar operaciones (unión, intersección, diferencia) para analizar colecciones de logros entre jugadores.

Crea una función gen_player_achievements() que use una lista grande y fija de logros para asignar aleatoriamente un conjunto a un jugador. Elige un número aleatorio de logros, luego selecciona esa cantidad de logros de la lista para construir y devolver el conjunto.

Entonces tu código hará:

- Generar conjuntos de logros para un mínimo de cuatro jugadores diferentes
- Rastrear logros únicos entre todos los jugadores
- Encontrar logros compartidos por todos los jugadores
- Para cada jugador, identificar los logros que nadie más tiene
- Para cada jugador, listar los logros faltantes para tenerlos todos

```
$> python3 ft_achievement_tracker.py
=== Sistema de Seguimiento de Logros ===

Jugador Alice: {'Genius de Fabricación', 'Salvador del Mundo', 'Maestro Explorador', 'Supremo Coleccionista', 'Intocable',
        'Asesino de Jefes'}
Jugador Bob: {'Genius de Fabricación', 'Estratega', 'Salvador del Mundo', 'Maestro Explorador', 'Imparable', '
        Supremo Coleccionista', 'Intocable'}
Jugador Charlie: {'Estratega', 'Corredor Rápido', 'Superviviente', 'Maestro Explorador', 'Cazador de Tesoros', 'Primer
        Pasos', 'Supremo Coleccionista', 'Intocable', 'Mente Afilada'}
Jugador Dylan: {'Estratega', 'Corredor Rápido', 'Imparable', 'Intocable', 'Asesino de Jefes'}

Todos los logros distintos: {'Genius de Fabricación', 'Estratega', 'Salvador del Mundo', 'Corredor Rápido', 'Superviviente',
        'Maestro Explorador', 'Cazador de Tesoros', 'Imparable', 'Primeros Pasos', 'Supremo Coleccionista', '
        'Intocable', 'Mente Afilada', 'Asesino de Jefes'}

Logros comunes: {'Intocable'}

Solo Alice tiene: conjunto()
Solo Bob tiene: conjunto()
Solo Charlie tiene: {'Superviviente', 'Cazador de Tesoros', 'Primeros Pasos', 'Mente Afilada'}
Solo Dylan tiene: conjunto()

Alice está perdiendo: {'Estratega', 'Corredor Rápido', 'Superviviente', 'Cazador de Tesoros', 'Imparable', 'Oculto
        Path Finder', 'Primeros Pasos', 'Mente Afilada'}
Bob está perdiendo: {'Corredor Rápido', 'Superviviente', 'Cazador de Tesoros', 'Oculto Path Finder', 'Primeros Pasos', '
        Mente Afilada', 'Asesino de Jefes'}
Charlie está perdiendo: {'Genius de Fabricación', 'Salvador del Mundo', 'Oculto Path Finder', 'Imparable', 'Jefe
        Asesino'}
Dylan está perdiendo: {'Genius de Fabricación', 'Salvador del Mundo', 'Superviviente', 'Maestro Explorador', 'Cazador de Tesoros',
        'Oculto Path Finder', 'Primeros Pasos', 'Supremo Coleccionista', 'Mente Afilada'}
```

Ajusta el número total de logros y cuántos recoges para cada jugador, para que todos los conjuntos solicitados probablemente no estén vacíos. Por cierto, ¿cómo imprime Python un conjunto vacío, y por qué?

Capítulo IX

Ejercicio 4: Maestro del Inventario

	Ejercicio4
ft_sistema_inventario	
Directorio: ex4/	
Archivos para enviar: ft_inventory_system.py	
Autorizado: importar sys, sys.argv, len(), print(), sum(), list(), round(), dict.keys(), dict.values(), dict.update()	

Tiempo de saqueo! ¿Recuerdas organizar tu inventario en los RPGs? ¿Comprobar si tienes la espada legendaria? ¡Es hora de construir el sistema de inventario definitivo!

Este ejercicio requiere el uso de diccionarios para almacenar datos del inventario.

Tu código primero analizará los parámetros de la línea de comandos para completar tu sistema de inventario. Cada parámetro debe seguir este formato: "<nombre_del_artículo>:<cantidad> . Descartar inválidos parámetros (sintaxis inválida, parámetros redundantes) con un mensaje de error, y colocar los válidos en un diccionario. Los valores de <quantity> en el diccionario se almacenarán como un int para que los cálculos se realicen más tarde.

Hora de operar con tu inventario:

- Muestra tu inventario
- Crea y muestra la lista de todos los artículos que forman parte de tu inventario
- Calcula e imprime la cantidad total de todos los artículos en el inventario
- Muestra para cada artículo el porcentaje de cantidad que representa en el inventario
- Informe los artículos más y menos abundantes (eligiendo el primero de la línea de comandos en caso de empate)
- Finalmente, añade un nuevo artículo a tu inventario y muéstralo de nuevo

```
$> python3 ft_inventory_system.py sword:1 potion:5 shield:2 armor:3 helmet:1 sword:2 hola clave:valor
=== Análisis del Sistema de Inventario ===
Elemento redundante 'espada' - descartando
Error - parámetro inválido 'hola'
Error de cantidad para 'key': literal inválido para int() con base 10: 'value'
Inventario obtenido: {'sword': 1, 'potion': 5, 'shield': 2, 'armor': 3, 'helmet': 1}
Lista de ítems: ['sword', 'potion', 'shield', 'armor', 'helmet']
Cantidad total de los 5 ítems: 12
El ítem espada representa 8.3%
El ítem poción representa 41.7%
El ítem escudo representa 16.7%
El ítem armadura representa 25.0%
El ítem casco representa 8.3%
Ítem más abundante: poción con cantidad 5
Ítem menos abundante: espada con cantidad 1
Inventario actualizado: {'sword': 1, 'potion': 5, 'shield': 2, 'armor': 3, 'helmet': 1, 'magic_item': 1}
```

Al comenzar el juego, tu inventario normalmente está vacío ;)

Capítulo X

Ejercicio 5: Mago de flujo

	Ejercicio5
ft_data_stream	
Directorio: ex5/	
Archivos para enviar: ft_data_stream.py	
Autorizado: next(), range(), len(), print(), import typing, typing.Generator, import random, random.*	

¡Hora Mágica! ¿Alguna vez te has preguntado cómo los juegos manejan millones de eventos sin fallar? Bienvenido al mundo de los generadores, el superpoder de ahorro de memoria de Python.

Este ejercicio requiere el uso de generadores con la palabra clave yield para crear flujos de datos al vuelo. Debes implementar funciones generadoras que produzcan valores bajo demanda en lugar de almacenar todo en la memoria.

Crea una función generadora infinita gen_event() que elija un nombre aleatorio de una lista de jugadores y una acción aleatoria de una lista de acciones. Cada vez que se llame a next() en este generador, devuelve un nuevo evento como una tupla (nombre, acción).

Desde la parte principal de tu script, haz un bucle de mil veces y muestra los 1000 eventos que obtienes de tu gen_event().

Luego crea una lista de diez tuplas generadas por gen_event() nuevamente.

Finalmente, crea una nueva función generadora consume_event que tome la lista creada previamente, elija aleatoriamente uno de sus elementos, lo elimine de la lista y lo produzca, hasta que la lista esté vacía. Este generador debe usarse directamente en la construcción for .. in ...

```
$> python3 ft_data_stream.py
=== Procesador de Flujo de Datos de Juego ===
Evento 0: El jugador bob realizó la acción correr
Evento 1: La jugadora alice realizó la acción comer
Evento 2: El jugador bob realizó la acción dormir
Evento 3: El jugador bob realizó la acción agarrar
Evento 4: El jugador dylan realizó la acción correr
Evento 5: El jugador bob realizó la acción mover
Evento 6: La jugadora alice realizó la acción mover
Evento 7: El jugador dylan realizó la acción mover
Evento 8: La jugadora alice realizó la acción escalar
Evento 9: El jugador bob realizó la acción dormir
Evento 10: El jugador bob realizó la acción correr
Evento 11: El jugador bob realizó la acción nadar
Evento 12: El jugador dylan realizó la acción nadar
Evento 13: El jugador charlie realizó la acción dormir
Evento 14: El jugador charlie realizó la acción dormir
```

[...]

```
Evento 992: El jugador dylan realizó la acción comer
Evento 993: La jugadora alice realizó la acción dormir
Evento 994: El jugador charlie realizó la acción mover
Evento 995: El jugador charlie realizó la acción escalar
Evento 996: El jugador bob realizó la acción soltar
Evento 997: El jugador bob realizó la acción agarrar
Evento 998: El jugador bob realizó la acción mover
Evento 999: La jugadora alice realizó la acción mover
Lista construida de 10 eventos: [('charlie', 'mover'), ('dylan', 'agarrar'), ('alice', 'usar'), ('alice', 'usar'), ('
    charlie', 'nadar'), ('bob', 'correr'), ('charlie', 'mover'), ('dylan', 'escalar'), ('alice', 'usar'), ('bob
    ', 'soltar')]
Evento obtenido de la lista: ('charlie', 'nadar')
Quedan en la lista: [('charlie', 'mover'), ('dylan', 'agarrar'), ('alice', 'usar'), ('alice', 'usar'), ('bob', '
    correr'), ('charlie', 'mover'), ('dylan', 'escalar'), ('alice', 'usar'), ('bob', 'soltar')]
Evento obtenido de la lista: ('alice', 'usar')
Permanece en la lista: [('charlie', 'move'), ('dylan', 'grab'), ('alice', 'use'), ('bob', 'run'), ('charlie',
    'move'), ('dylan', 'climb'), ('alice', 'use'), ('bob', 'release')]
Obtente del listado: ('charlie', 'move')
Permanece en la lista: [('dylan', 'grab'), ('alice', 'use'), ('bob', 'run'), ('charlie', 'move'), ('dylan', '
    climb'), ('alice', 'use'), ('bob', 'release')]
Obtente del listado: ('charlie', 'move')
Permanece en la lista: [('dylan', 'grab'), ('alice', 'use'), ('bob', 'run'), ('dylan', 'climb'), ('alice',
    use'), ('bob', 'release')]
Obtente del listado: ('alice', 'use')
Permanece en la lista: [('dylan', 'grab'), ('bob', 'run'), ('dylan', 'climb'), ('alice', 'use'), ('bob',
    release')]
Obtente del listado: ('bob', 'release')
Permanece en la lista: [('dylan', 'grab'), ('bob', 'run'), ('dylan', 'climb'), ('alice', 'use')]
Obtente del listado: ('bob', 'run')
Permanece en la lista: [('dylan', 'grab'), ('dylan', 'climb'), ('alice', 'use')]
Obtente del listado: ('dylan', 'climb')
Permanece en la lista: [('dylan', 'grab'), ('alice', 'use')]
Obtente del listado: ('alice', 'use')
Permanece en la lista: [('dylan', 'grab')]
Obtente del listado: ('dylan', 'grab')
Permanece en la lista: []
```

Capítulo XI

Ejercicio 6: Alquimista de Datos

	Ejercicio6
ft_data_alchemist	
Directorio: ex6/	
Archivos a entregar: ft_data_alchemist.py	
Autorizado: import random, random.*, print(), len(), sum(), round()	

¡Hora del jefe final! Has dominado todas las estructuras de datos. ¡Ahora es momento de descubrirlas en una forma elegante y condensada! ¡Aquí es donde te conviertes en un verdadero Alquimista de Datos!

Este ejercicio requiere el uso de comprensiones de listas y diccionarios para transformar y filtrar datos eficientemente. Estas son características fundamentales de Python para el procesamiento de datos.

Crea una lista de nombres de jugadores, donde algunos están en mayúscula y otros no. Construye dos comprensiones de listas: la primera crea una nueva lista con todos los nombres en mayúscula, la segunda crea una nueva lista solo con los nombres que ya estaban en mayúscula de la lista inicial.

Ahora, vamos a crear un diccionario a partir de esta lista completamente en mayúscula de nombres de jugadores. Los nombres serán las claves y los valores serán puntuaciones generadas aleatoriamente en un rango definido. Por supuesto, una comprensión construirá este diccionario. Luego se creará un segundo diccionario, con puntuaciones superiores al promedio, también usando una comprensión.

```
$> python3 ft_data_alchemist.py ===
Juego Alquimista de Datos ===

Lista inicial de jugadores: ['Alice', 'bob', 'Charlie', 'dylan', 'Emma', 'Gregory', 'john', 'kevin', 'Liam
        ']
Nueva lista con todos los nombres en mayúscula: ['Alice', 'Bob', 'Charlie', 'Dylan', 'Emma', 'Gregory', 'John', '
        Kevin', 'Liam']
Nueva lista solo con los nombres en mayúscula: ['Alice', 'Charlie', 'Emma', 'Gregory', 'Liam']

Diccionario de puntuaciones: {'Alice': 263, 'Bob': 666, 'Charlie': 907, 'Dylan': 170, 'Emma': 568, 'Gregory': 446, 'John
        ': 90, 'Kevin': 527, 'Liam': 54}
El promedio de puntuaciones es 410.11
Puntuaciones altas: {'Bob': 666, 'Charlie': 907, 'Emma': 568, 'Gregory': 446, 'Kevin': 527}
```

También es posible usar comprensión en conjuntos.

Cada comprensión debe estar en una sola línea (a menos que supere el tamaño de la línea).

Capítulo XII Entrega y Presentación

Entrega tu tarea en tu repositorio Git como de costumbre. Solo el trabajo dentro de tu repositorio será evaluado durante la defensa. No dudes en verificar dos veces los nombres de tus archivos para asegurarte de que son correctos.

Durante la evaluación, puede que se te pida explicar las opciones de estructuras de datos, demostrar operaciones de colecciones o ampliar tus sistemas de análisis con nueva funcionalidad. Asegúrate de entender los principios detrás de cada estructura de datos.

Necesitas devolver solo los archivos solicitados por el tema de este proyecto. Enfócate en código limpio y bien documentado que demuestre claramente dominio de los tipos de colección de Python y técnicas de procesamiento de datos.